Department of Computer Science and Engineering

Item	Details
Course Code & Name	CSA0617 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192511209
Student Name	Jason Joshva M J

Assumptions used throughout. The servers execute 10^8 basic operations per second, so time = operations ÷ 10^8 seconds. The catalogue grows from n = 100,000 to n = 1,000,000 products. For the recurrences, the base case costs one operation, T (1) = 1; log means $\log_2$.

Audit Summary

At n = 1,000,000 three nightly jobs exceed the 10-minute window: **M1 duplicate detector (83 min)**, **M4 recommendation engine in its current design (5.6 h)** and the **report generator inside M5 (8.3 h)** — all three are $\Theta(n^2)$ and all three still pass today. M2 (10.5 ms), binary search S2 (0.2 µs per search), the M4 redesign (0.21 s) and the M5 analytics engine (2.1 s) scale safely. **Redesign M1 (hash-based duplicate detection), adopt the proposed M4 redesign, and replace the quadratic step of the report generator**; retain M2, S2 and the M5 analytics engine. The team member's (n log n) claim for M2 is wrong: M2 is $\Theta(n)$.

(a) Understand the Problem

The **basic operation** of each module is the step inside its innermost loop or recursion whose count grows fastest with n. Every other step — loop control, index updates, function-call overhead — runs at most a constant number of times per basic operation, so the running time is a constant multiple of the basic-operation count.

Table A.1: Input size, basic operation and expected growth of each module

Module	Input size n	Basic operation	Where it occurs	Expected growth when n grows 10×
M1	Number of products in the catalogue	Comparing two listings: Product[i].title = Product[j].title (the seller is compared only when the titles match)	The IF inside the inner loop — once for every pair (i, j) with i < j	Quadratic: n(n−1)/2 pairs, so about **100**× more work

M2	Number of requests n (size of the largest window)	Audit(Request[j])	The inner FOR loop, which runs i times for each window size i = 1, 2, 4, …	Linear: fewer than 2n audits, so about **10×**

Module	**Input size n**	**Basic operation**	**Where it occurs**	**Expected growth when n grows 10×**
M3 – S1	Number of products in the unsorted catalogue	Comparing the search key with one product	The scan loop — once per product examined	Linear: up to n comparisons, **10×**
M3 – S2	Number of entries in the sorted index	Comparing the key with the middle entry	Once per halving step (the +1 in T(n) = T(n/2) + 1)	Logarithmic: only about **3.3 more** comparisons
M4	Size of the customer–product data	One unit of merge work (the + n term)	The linear-time merge at every level of the recursion	Current $\Theta(n^2)$: about **100×**; redesign $\Theta(n \log n)$: about **12×**
M5 – analytics	Number of sales records	One comparison while sorting by revenue (the + n log n term)	The sort performed at every level of the recursion	$\Theta(n \log^2 n)$: about **14×**
M5 – report	Number of sales records	One profiled operation; dominated by the $3n^2$ term	A pairwise (nested-loop) step over the records	Quadratic: about **100×**

Why these choices. For M1 every product is compared with every later product, so the comparison is executed once per pair. For M2 the audit is the only work inside the inner loop, and the outer loop only doubles i. For S1 and S2 the key comparison decides whether to stop or continue. For M4 and M5 the recurrences already state the non-recursive work per call (n and n log n), so that work is the basic operation, summed over every level of the recursion.

(b) Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

For a fixed i, the inner loop runs for j = i + 1, …, n − 1, which is (n − 1 − i) comparisons. Summing over every i:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

Exact count: C(n) = n(n − 1)/2 comparisons. Time: $\Theta(n^2)$. Space: $\Theta(1)$ extra.

- The loops never stop early, so the count is the same for every input: best = average = worst = n(n − 1)/2. If the seller comparison in the AND is counted separately, it runs only when two titles match; even if every title matched it would at most double the count, which is still $\Theta(n^2)$.

- Extra space is Θ(1): only the indices i and j are needed, and MarkDuplicate sets a flag in the existing record.
- **Today (n = 100,000):** 4,999,950,000 comparisons → 50.0 s. **At n = 1,000,000:** 499,999,500,000 comparisons → 5,000 s = **83.3 minutes**. Ten times more products means one hundred times more work.

M2 – API Rate Limiter

The outer loop runs with i = 1, 2, 4, …, 2^t while i < n, and for each value the inner loop calls Audit exactly **i** times (j = 0 … i − 1), not n times. Let K be the last t with $2^t < n$, so $K = \lceil \log_2 n \rceil - 1$. The total is a geometric series:

$$A(n) = \sum_{t=0}^{K} 2^t = 1 + 2 + 4 + \cdots + 2^K = 2^{K+1} - 1 = 2^{\lceil \log_2 n \rceil} - 1$$

Because $n \le 2^{\lceil \log_2 n \rceil} < 2n$:

$$n - 1 \le A(n) < 2n - 1 \quad \Rightarrow \quad A(n) = \Theta(n)$$

Exact count: $A(n) = 2^{\lceil \log_2 n \rceil} - 1$ audits (= n − 1 when n is a power of two). Tight bound: Θ(n). Space: Θ(1).

Is the Θ(n log n) claim correct? No. The claim multiplies the number of outer iterations (about log n) by the largest inner count (n). But the inner count is i, which doubles each time: only the last iteration comes close to n, and all the earlier iterations together add up to less than the last one ($1 + 2 + \ldots + 2^{K-1} = 2^K - 1 < 2^K$). So n log n is a valid **upper** bound — M2 is O(n log n) — but not a tight one. The tight bound is **Θ(n)**.

Table B.1: Verification for n = 1,024

Iteration t	**1**	**2**	**3**	**4**	**5**	**…**
$i = 2^{t-1}$	1	2	4	8	16	32, 64, 128, 256, 512
Audit calls	1	2	4	8	16	32 + 64 + 128 + 256 + 512

The loop runs 10 times (i = 1 … 512) and stops when i = 1,024, because 1,024 < 1,024 is false. Total audits = 1 + 2 + 4 + … + 512 = **1,023** = $2^{10} - 1$ = n − 1, exactly as the formula predicts. The claimed Θ(n log n) would predict about 1,024 × $\log_2$ 1,024 = **10,240** — ten times too many. At n = 1,000,000: $2^{20} - 1$ = 1,048,575 audits → **10.5 ms**.

(c) Solve a Recurrence by Substitution — Search Version S2

S2 compares the key with the middle entry (1 comparison) and continues in one half, so T(n) = T(n/2) + 1 with T(1) = 1. Backward substitution replaces T on the right-hand side by its own definition, one step at a time:

$$\text{Step } 1: \quad T(n) = T(n/2) + 1$$

$$\text{Step } 2: \quad T(n) = [\, T(n/4) + 1 \,] + 1 = T(n/4) + 2$$

$$\text{Step } 3: \quad T(n) = [\, T(n/8) + 1 \,] + 2 = T(n/8) + 3$$

Each step halves the argument once more and adds one more comparison. After k steps the pattern is:

$$\text{Step } k: \quad T(n) = T(n/2^k) + k$$

The substitution stops at the base case, when the subproblem has one entry: $n/2^k = 1$, that is $2^k = n$, so $k = \log_2 n$ (n a power of two). Substituting k:

$$T(n) = T(1) + \log_2 n = \log_2 n + 1$$

Closed form: T(n) = $\log_2$ n + 1 (for any n, with T(n) = T(⌊n/2⌋) + 1: T(n) = ⌊$\log_2$ n⌋ + 1). Time: Θ(log n).

Check: T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 4 = $\log_2$ 8 + 1. At n = 1,000,000: ⌊19.93⌋ + 1 = **20 comparisons** (today 17). Ten times more products adds only about 3.3 comparisons.

Table C.1: Space complexity of S2

Implementation	Space	Why
Iterative	Θ(1)	Only low, high and mid are kept, and they are overwritten at every step.
Recursive	Θ(log n)	Each call keeps its own frame (low, high, mid, return address) on the stack until it returns, and the calls nest ⌊$\log_2$ n⌋ + 1 deep — 20 frames at n = 10^6. Because the recursive call is the last action (tail recursion), a compiler that removes tail calls can bring this down to Θ(1).

(d) Apply the Master Theorem

The Master Theorem solves T(n) = aT(n/b) + f(n), with a ≥ 1 and b > 1, by comparing f(n) with n^(log_b a): **Case 1**, f(n) = O(n^(log_b a − ε)) gives Θ(n^(log_b a)); **Case 2**, f(n) = Θ(n^(log_b a)) gives Θ(n^(log_b a) log n); **Case 3**, f(n) = Ω(n^(log_b a + ε)) with the regularity condition gives Θ(f(n)).

M4 – Recommendation Engine

Table D.1: Master Theorem for both designs of M4

	Current design	**Proposed redesign**
Recurrence	$T(n) = 4T(n/2) + n$	$T(n) = 2T(n/2) + n$
a, b	a = 4, b = 2	a = 2, b = 2
f(n)	n	n
n^(log_b a)	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	$f(n) = n = O(n^{2-\varepsilon})$ with $\varepsilon = 1$: f grows polynomially slower	$f(n) = n = \Theta(n^{(\log_b a)})$
Case	**Case 1**	**Case 2**
Result	**$T(n) = \Theta(n^2)$**	**$T(n) = \Theta(n \log n)$**
Exact, with T(1) = 1	$T(n) = 2n^2 - n$	$T(n) = n \log_2 n + n$
When the data doubles	$T(2n) \approx 4 \cdot T(n)$ — four times as long	$T(2n) = 2T(n) + 2n \approx 2.1 \cdot T(n)$ at $n = 10^6$
Operations at $n = 10^6$	$\approx 2.0 \times 10^{12} \rightarrow$ 20,000 s ≈ **5.6 h**	$\approx 2.09 \times 10^7 \rightarrow$ **0.21 s**

Where the exact forms come from (recursion tree). In the current design level i has 4^i subproblems of size $n/2^i$, so it costs $4^i \cdot n/2^i = 2^i n$; with $L = \log_2 n$ levels and $4^L = n^2$ leaves:

$$T(n) = \sum_{i=0}^{L-1} 2^i n + 4^L T(1) = n(2^L - 1) + n^2 = 2n^2 - n$$

In the redesign every level costs $2^i \cdot n/2^i = n$, so $T(n) = n \log_2 n + n \cdot T(1) = n \log_2 n + n$.

What doubling means. In the current design the four half-size subproblems make the number of leaves grow as n^2, so doubling the data quadruples the run time. Removing the replication halves the branching: doubling the data then adds only one level of recursion — one extra pass of n — so the time roughly doubles. At $n = 10^6$ the redesign is about **95,500 times faster**.

M5 – Sales Analytics Engine: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, f(n) = n log n and $n^{(\log_b a)} = n^{\log_2 2} = n$. Checking the three standard cases:

- **Case 1** needs $f(n) = O(n^{1-\varepsilon})$: no — n log n grows faster than n.
- **Case 2** needs $f(n) = \Theta(n)$: no — $f(n)/n = \log n \rightarrow \infty$.
- **Case 3** needs $f(n) = \Omega(n^{1+\varepsilon})$ for some $\varepsilon > 0$: no — $f(n)/n^{1+\varepsilon} = \log n / n^{\varepsilon} \rightarrow 0$ for every $\varepsilon > 0$. So f(n) is larger than n, but only by a logarithmic factor, not by a polynomial factor n^{ε}: it falls in the gap between Cases 2 and 3, and **none of the three standard cases applies**.

Extended Master Theorem (Case 2 with a log factor): if $f(n) = \Theta(n^{(\log_b a)} \cdot \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{(\log_b a)} \cdot \log^{k+1} n)$. Here $f(n) = \Theta(n \cdot \log^1 n)$, so k = 1 and:

$$T(n) = \Theta(n\log^2 n)$$

Confirmation by recursion tree (L = $\log_2 n$ levels):

Table D.2: Recursion tree for M5

Level i	Subproblems	Size of each	Cost of each	Cost of the level
0	1	n	$n \log_2 n$	$n \log_2 n$
1	2	n/2	$(n/2) \log_2(n/2)$	$n (\log_2 n - 1)$
2	4	n/4	$(n/4) \log_2(n/4)$	$n (\log_2 n - 2)$
i	2^i	$n/2^i$	$(n/2^i) \log_2(n/2^i)$	$n (\log_2 n - i)$
L (leaves)	n	1	T(1) = 1	n

$$T(n) = \sum_{i=0}^{L-1} n\,(L-i) + n = n\,\frac{L(L+1)}{2} + n = \frac{n\log_2 n\,(\log_2 n + 1)}{2} + n = \Theta(n\log^2 n)$$

Check: T(4) = 2T(2) + 4·2 = 2(2·1 + 2) + 8 = 16, and the formula gives 4·2·3/2 + 4 = 16. At $n = 10^6$ ($\log_2 n = 19.93$): $\approx 2.10 \times 10^8$ operations → **2.1 s** (today 0.15 s). It is only a log factor above n log n,

so ten times more data costs about 14 times more time. Sorting once and passing sorted halves up the recursion (as merge sort does) would make it Θ(n log n).

(e) Analyse Best, Average and Worst Cases — S1 and S2

Table E.1: Input conditions and comparison counts

Case	S1 (sequential scan): input condition	S1 comparisons	S2 (halving the sorted index): input condition	S2 comparisons
Best	Key is the first product scanned	1	Key equals the first middle entry	1
Average	Key present and equally likely to be at any of the n positions	(n + 1)/2	Key present and equally likely to be any entry	$\approx \log_2(n + 1) - 1$
Worst	Key is the last product, or not in the catalogue	n	Key absent, or found only at the last halving step	$\lfloor \log_2 n \rfloor + 1$
$n = 10^6$		1 / 500,000.5 / 1,000,000		1 / ≈ 18.9 / 20

Why the cases differ. For S1 the cost equals the position of the key, because it checks products one by one in an unsorted catalogue and cannot stop until it finds the key or reaches the end. For S2 the

cost equals the depth of the key in the halving decision tree; half of all entries sit on the deepest level, so the average is only about one comparison below the worst case. For S2 the average comes from the decision tree (for $n = 2^h - 1$):

$$C_{\text{avg}}^{S2}(n) = \frac{1}{n}\sum_{j=1}^{h} j \cdot 2^{j-1} = \frac{(h-1)2^h + 1}{n} \approx h - 1 = \log_2(n+1) - 1$$

S1 average when the product exists with probability p

Found (probability p): the key is at position i with probability 1/n and costs i comparisons, so the expected cost is (1/n)(1 + 2 + … + n) = (n + 1)/2. **Not found** (probability 1 − p): all n products are compared. Combining the two:

$$C_{\text{avg}}^{S1}(n) = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

For **n = 1,000,000 and p = 0.5**:

$$C_{\text{avg}}^{S1} = 0.5 \times \frac{1{,}000{,}001}{2} + 0.5 \times 1{,}000{,}000 = 250{,}000.25 + 500{,}000 = 750{,}000.25$$

S1 averages ≈ 750,000 comparisons (7.5 ms) per search; S2 needs at most 20 (0.2 µs) — about 37,500 times fewer.

What this means for ShopSphere. At 10^8 operations per second, one server core can answer only about **133 S1 searches per second** at 1,000,000 products, against about **5,000,000** with S2. Live product search must use S2 (or a hash index); S1 is acceptable only for rare, one-off lookups.

(f) Prove an Asymptotic Bound

Definition. $f(n) = \Theta(n^2)$ if there are positive constants c_1, c_2 and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for every $n \ge n_0$. Here $f(n) = 3n^2 + 10n \log_2 n + 50$.

Lower bound. For $n \ge 1$, $\log_2 n \ge 0$, so $10n \log_2 n \ge 0$; also $50 > 0$. Dropping both terms can only make the expression smaller:

$$f(n) \ \ge \ 3n^2 \qquad \text{for all } n \ge 1 \qquad (c_1 = 3)$$

Upper bound. For $n \ge 1$, $\log_2 n \le n$, so $10n \log_2 n \le 10n^2$; and $50 \le 50n^2$. Therefore:

With

$c_1 = 3$, $c_2 = 63$ and $n_0 = 1$: $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$, so $f(n) = \Theta(n^2)$.

Check at n = 1: f(1) = 3 + 0 + 50 = 53, and $3 \le 53 \le 63$. A tighter upper constant also works with a larger n_0: $c_2 = 4$ for $n \ge 60$, because $10n \log_2 n + 50 \le n^2$ from n = 60 on (at n = 60: 3,594.1 ≤ 3,600).

f(n) is not O(n). Suppose it were: then some $c > 0$ and n_0 would give $f(n) \le cn$ for all $n \ge n_0$. Since $f(n) \ge 3n^2$, this means $3n^2 \le cn$, that is $n \le c/3$, for all $n \ge n_0$ — which is false for any $n > \max(n_0, c/3)$. This contradiction shows $f(n) \ne O(n)$. Equivalently, $f(n)/n \ge 3n \to \infty$.

What this means for the platform. Only the $3n^2$ term matters. At $n = 10^6$ it contributes 3×10^{12} operations, while $10n \log_2 n$ contributes about 2×10^8 — less than 0.01% of the total. The report takes about **8.3 hours**, and tuning the lower-order terms cannot fix that: the quadratic step itself must be removed.

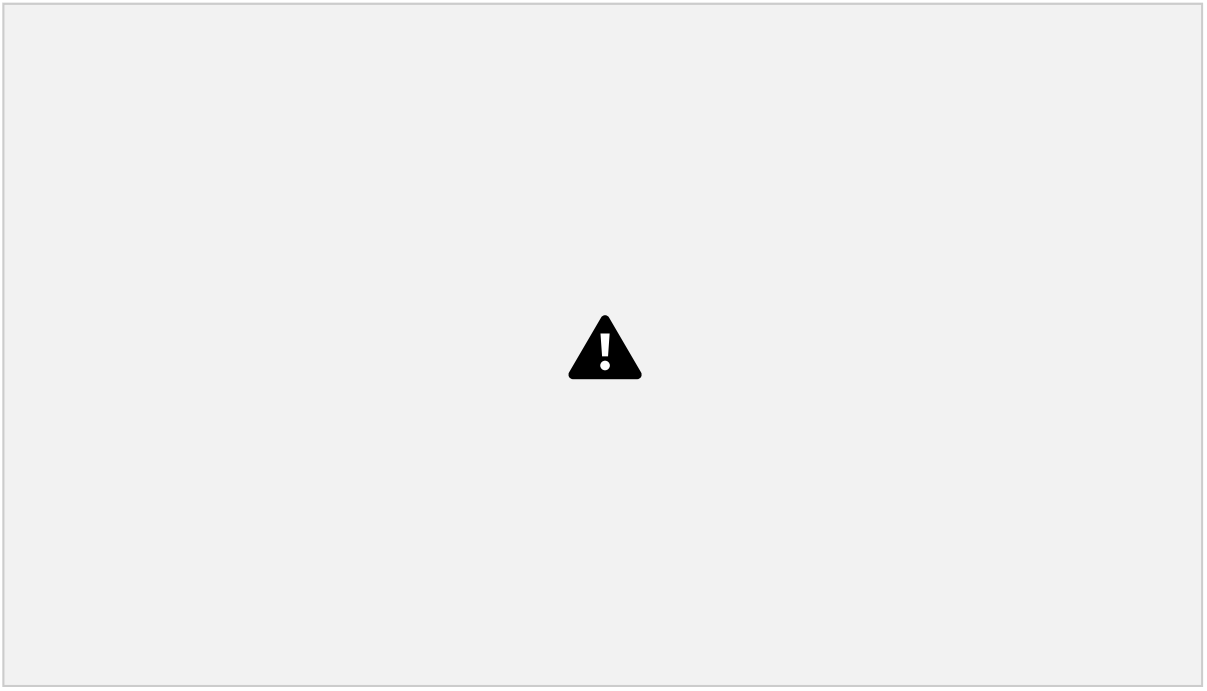
(g) Scalability Decision

A 10-minute window allows $600 \text{ s} \times 10^8 = \mathbf{6 \times 10^{10}}$ **basic operations**. Table G.1 applies the exact counts derived above.

Table G.1: Estimated operations and running time (10^8 operations per second)

Module	Complexity	Ops at $n = 10^5$	Time today	Ops at $n = 10^6$	Time at 10^6	Fits 10 min?
M1 duplicate detector	$\Theta(n^2)$	5.0×10^9	50.0 s	5.0×10^{11}	5,000 s = 83.3 min	**No**
M2 rate limiter	$\Theta(n)$	131,071	1.3 ms	1,048,575	10.5 ms	Yes
S2 binary search (per search)	$\Theta(\log n)$	17	0.17 µs	20	0.20 µs	Yes
M4 current	$\Theta(n^2)$	2.0×10^{10}	200 s = 3.3 min	2.0×10^{12}	20,000 s = 5.6 h	**No**
M4 redesign	$\Theta(n \log n)$	1.76×10^6	17.6 ms	2.09×10^7	0.21 s	Yes
M5 analytics engine	$\Theta(n \log^2 n)$	1.47×10^7	0.15 s	2.10×10^8	2.1 s	Yes
M5 report generator	$\Theta(n^2)$	3.0×10^{10}	300 s = 5.0 min	3.0×10^{12}	30,002 s = 8.3 h	**No**

Figure G.1: Estimated running time today and at 1,000,000 products, against the 10-minute nightly window



Modules that fail: M1, M4 (current design) and the M5 report generator. All three are quadratic and all three pass today; a 10× larger catalogue multiplies their work by 100. Their breaking points come long before 1,000,000 products: $3n^2 \leq 6 \times 10^{10}$ fails beyond **n ≈ 141,000** (report generator), $2n^2$ beyond **n ≈ 173,000** (M4) and $n^2/2$ beyond **n ≈ 346,000** (M1). The report generator will miss the window once the catalogue grows by about 41%.

Table G.2: Recommendations

Module	Decision	How	New cost at $n = 10^6$
M1 duplicate detector	**Redesign**	Use a hash set keyed by (title, seller): one pass over the catalogue, inserting each product; a key already in the set is a duplicate. Expected Θ(n) time, Θ(n) extra space. Alternative: sort by (seller, title) in Θ(n log n), then compare neighbours only. Better still, check each new upload against the set when it is uploaded (Θ(1) per upload), so the nightly pass is not needed at all.	≈ 10^6 hash operations (sorting: ≈ 2×10^7 comparisons) → under 0.2 s
M4 recommendation engine	**Redesign** (adopt the proposal)	Remove the replicated partitions: 2 subproblems instead of 4 turns $\Theta(n^2)$ into Θ(n log n) (Case 1 → Case 2).	2.09×10^7 → 0.21 s (≈ 95,500× faster)
M5 report generator	**Redesign**	Find the step behind the $3n^2$ term — a nested loop over all pairs of records — and replace it with hash-based grouping (Θ(n)) or sort-then-scan (Θ(n log n)); keep running totals updated as each sale arrives.	≤ 2×10^7 → under 0.2 s
M5 analytics engine	Retain	2.1 s fits easily. Optional: sort once and merge sorted halves to reach Θ(n log n).	2.1 s (≈ 0.2 s if improved)
M2 rate limiter	Retain	It is Θ(n), not Θ(n log n): about 10^6 audits.	10.5 ms
M3 product search	Use S2; retire S1	S2 needs at most 20 comparisons. Keep the index sorted by merging each day's new products into it instead of re sorting.	0.2 µs per search

Justification. After these changes the whole nightly batch — M1, M4, the M5 analytics engine and

the report generator — needs about 2.7 s, well within the 600 s window, and every nightly job is $O(n \log^2 n)$ or better, so even a further tenfold growth to 10^7 products would still finish in under a minute. Without the changes the three quadratic jobs together would need about **15.3 hours** at 1,000,000 products, more than the whole night.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I ***Jason Joshva M J / 192511209***(Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Jason Joshva M J* Date: 12/10/2026